

Sequence Parallelism: A Concrete Walkthrough with Actual Numbers

1. The Problem: What Tensor Parallelism Leaves on the Table

In our Tensor Parallelism walkthrough, we saw that TP splits the large weight matrices across GPUs, giving us $\sim 1/N$ memory for model parameters. But we also noted a frustrating limitation:

Operations PARALLELIZED by TP:	Operations NOT parallelized by TP:
• W_q, W_k, W_v (column-parallel)	• LayerNorm (needs full d_{model})
• W_o (row-parallel)	• Dropout (needs full d_{model})
• $W1$ (column-parallel)	• Residual connections (full d_{model})
• $W2$ (row-parallel)	• Activation storage between layers

The "not parallelized" operations all require the **full hidden dimension** on every GPU, which means their activations are **replicated** — we don't get any memory savings for them.

Sequence Parallelism (SP) fixes this by splitting along the **sequence dimension** instead of the hidden dimension for these operations.

2. Our Tiny Transformer Setup (Same as Previous Walkthroughs)

Hidden dimension (d_{model}) = 4
Number of attention heads = 2 $\rightarrow$ head dimension $d_k = 4/2 = 2$
FFN inner dimension (d_{ff}) = 16 (4 $\times$ expansion)
Vocab size = 8
Sequence length (T) = 4 (4 tokens — easily divisible by 2)
Batch size (B) = 2
Number of GPUs (TP degree) = 2 (GPU-0 and GPU-1)

Our activation tensors have shape **(B, T, d_{model}) = (2, 4, 4)**.

3. The Key Insight: Swapping All-Reduce for Reduce-Scatter + All-Gather

Vanilla TP Communication Pattern:

After row-parallel linear (e.g., W_o or $W2$):
GPU-0 has: partial_0 (B, T, d_{model}) = (2, 4, 4)
GPU-1 has: partial_1 (B, T, d_{model}) = (2, 4, 4)

ALL-REDUCE: sum partials $\rightarrow$ both GPUs get full (2, 4, 4)

LayerNorm: operates on full (2, 4, 4) on BOTH GPUs ← REDUNDANT!

Dropout: operates on full (2, 4, 4) on BOTH GPUs ← REDUNDANT!

TP + Sequence Parallelism Communication Pattern:

After row-parallel linear (e.g., W_o or W_2):

GPU-0 has: $\text{partial}_0 (B, T, d_{\text{model}}) = (2, 4, 4)$

GPU-1 has: $\text{partial}_1 (B, T, d_{\text{model}}) = (2, 4, 4)$

REDUCE-SCATTER: sum partials AND scatter along sequence dimension

GPU-0 gets: summed tokens 0-1 $\rightarrow (2, 2, 4)$ ← half the tokens!

GPU-1 gets: summed tokens 2-3 $\rightarrow (2, 2, 4)$ ← half the tokens!

LayerNorm: operates on (2, 2, 4) per GPU ← HALF THE MEMORY!

Dropout: operates on (2, 2, 4) per GPU ← HALF THE MEMORY!

Before next column-parallel linear:

ALL-GATHER: reconstruct full sequence

Both GPUs get: (2, 4, 4)

The total communication volume is identical — we just reorganized it:

- Vanilla TP: all-reduce = reduce-scatter + all-gather (fused)
- TP + SP: reduce-scatter (after row-linear) + all-gather (before column-linear)

Same bytes transferred, but now LayerNorm and Dropout operate on $1/N$ of the data!

4. Concrete Activation Shapes Throughout a Transformer Block

Let's trace the exact shapes on each GPU for both vanilla TP and TP+SP.

Notation:

- $\boxed{h}$: hidden dimension ($d_{\text{model}} = 4$)
- $\boxed{s}$: sequence length ($T = 4$)
- $\boxed{h/N}$: hidden dimension sharded ($= 2$ per GPU)
- $\boxed{s/N}$: sequence dimension sharded ($= 2$ per GPU)

Vanilla TP (No Sequence Parallelism):

Step	GPU-0 Shape	GPU-1 Shape	Note
------	-------------	-------------	------

Input to block	(2, 4, 4) (B, s, h)	(2, 4, 4) (B, s, h)	full, replicated
LayerNorm 1	(2, 4, 4)	(2, 4, 4)	full, replicated ✗
Enter TP: W _q , W _k , W _v (column-parallel)	(2, 4, 2) (B, s, h/N)	(2, 4, 2) (B, s, h/N)	h sharded (h/N=2) s full
Attention computation	(2, 4, 2)	(2, 4, 2)	local heads only
Exit TP: W _o (row-parallel + ALL-REDUCE)	(2, 4, 4) (B, s, h)	(2, 4, 4) (B, s, h)	h restored via ALL-REDUCE
Residual add	(2, 4, 4)	(2, 4, 4)	full, replicated ✗
LayerNorm 2	(2, 4, 4)	(2, 4, 4)	full, replicated ✗
Enter TP: W1 (column-parallel)	(2, 4, 8) (B, s, d_ff/N)	(2, 4, 8) (B, s, d_ff/N)	d_ff sharded (16/2=8)
GeLU	(2, 4, 8)	(2, 4, 8)	local
Exit TP: W2 (row-parallel + ALL-REDUCE)	(2, 4, 4) (B, s, h)	(2, 4, 4) (B, s, h)	h restored via ALL-REDUCE
Dropout	(2, 4, 4)	(2, 4, 4)	full, replicated ✗
Residual add	(2, 4, 4)	(2, 4, 4)	full, replicated ✗
Output of block	(2, 4, 4)	(2, 4, 4)	full, replicated
Peak activation (per GPU): (2, 4, 8) in FFN = 64 elements Wasted memory: LayerNorm, Dropout, Residuals all store full (2,4,4)=32 each			

TP + Sequence Parallelism:

Step	GPU-0 Shape	GPU-1 Shape	Note
Input to block (sequence-parallel region)	(2, 2, 4) (B, s/N, h)	(2, 2, 4) (B, s/N, h)	s sharded! (s/N=2) tokens 0-1 tokens 2-3
LayerNorm 1 (operates on local tokens)	(2, 2, 4) (B, s/N, h)	(2, 2, 4) (B, s/N, h)	s sharded ✓ SAVED!
ALL-GATHER before TP (SP → TP transition)	(2, 4, 4) (B, s, h)	(2, 4, 4) (B, s, h)	reconstruct full seq for column-parallel

Enter TP: W_q, W_k, W_v $(2, 4, 2)$ $(2, 4, 2)$ h sharded
(column-parallel) $(B, s, h/N)$ $(B, s, h/N)$ s full (needed for attn)

Attention computation $(2, 4, 2)$ $(2, 4, 2)$ local heads only

Exit TP: W_o
(row-parallel + REDUCE-SCATTER)
 $(2, 2, 4)$ $(2, 2, 4)$ s sharded again!
(TP $\rightarrow$ SP transition) $(B, s/N, h)$ $(B, s/N, h)$ tokens 0-1 | tokens 2-3

Residual add $(2, 2, 4)$ $(2, 2, 4)$ s sharded ✓ SAVED!

LayerNorm 2 $(2, 2, 4)$ $(2, 2, 4)$ s sharded ✓ SAVED!

ALL-GATHER before TP $(2, 4, 4)$ $(2, 4, 4)$ reconstruct full seq

Enter TP: $W1$ $(2, 4, 8)$ $(2, 4, 8)$ d_{ff} sharded
(column-parallel) $(B, s, d_{ff}/N)$ $(B, s, d_{ff}/N)$

GeLU $(2, 4, 8)$ $(2, 4, 8)$ local

Exit TP: $W2$
(row-parallel + REDUCE-SCATTER)
 $(2, 2, 4)$ $(2, 2, 4)$ s sharded again!

Dropout $(2, 2, 4)$ $(2, 2, 4)$ s sharded ✓ SAVED!

Residual add $(2, 2, 4)$ $(2, 2, 4)$ s sharded ✓ SAVED!

Output of block $(2, 2, 4)$ $(2, 2, 4)$ s sharded

Peak activation (per GPU): $(2, 4, 8)$ in FFN = 64 elements (same as vanilla TP)
BUT: LayerNorm, Dropout, Residuals now store $(2,2,4)=16$ each instead of 32!

5. Concrete Numbers: Walking Through the FFN with SP

Let's trace actual values through the FFN to see how reduce-scatter and all-gather work.

Input to FFN (after LayerNorm 2, in SP region):

GPU-0 has tokens 0-1:	GPU-1 has tokens 2-3:
$x_0 = \begin{bmatrix} 0.5, -0.3, 0.8, 0.1 \end{bmatrix},$	$x_1 = \begin{bmatrix} -0.6, 0.3, 0.5, -0.2 \end{bmatrix},$
$\begin{bmatrix} 0.2, 0.7, -0.1, 0.4 \end{bmatrix}$	$\begin{bmatrix} 0.4, -0.5, 0.3, 0.6 \end{bmatrix}$

Shape: (2, 2, 4) = (B, s/N, h)	Shape: (2, 2, 4)
(batch 0: tokens 0,1)	(batch 0: tokens 2,3)
(batch 1: tokens 0,1)	(batch 1: tokens 2,3)

Step 1: ALL-GATHER to Reconstruct Full Sequence (SP → TP)

Before the column-parallel W1, we need the full sequence on each GPU:

ALL-GATHER along sequence dimension:

GPU-0 sends x_0 (tokens 0-1) → GPU-1

GPU-1 sends x_1 (tokens 2-3) → GPU-0

After all-gather, BOTH GPUs have:

$x_{full} = [[0.5, -0.3, 0.8, 0.1], \leftarrow \text{token 0}$

$[0.2, 0.7, -0.1, 0.4], \leftarrow \text{token 1}$

$[-0.6, 0.3, 0.5, -0.2], \leftarrow \text{token 2}$

$[0.4, -0.5, 0.3, 0.6]] \leftarrow \text{token 3}$

Shape: (2, 4, 4) = (B, s, h)

Communication: each GPU sends (2, 2, 4) = 16 elements

Total all-gather volume: $2 \times 16 \times 2 \text{ bytes} = 64 \text{ bytes (BF16)}$

Step 2: Column-Parallel W1 (TP Region)

Each GPU computes with its slice of W1 (columns 0-7 or 8-15):

GPU-0: $h^{(0)} = x_{full} \cdot W_1^{(0)} \rightarrow (2, 4, 8) \text{ [d_ff/2 = 8]}$

GPU-1: $h^{(1)} = x_{full} \cdot W_1^{(1)} \rightarrow (2, 4, 8)$

No communication — each GPU processes all 4 tokens but only 8 of 16 FFN dims

Step 3: GeLU (Independent)

GPU-0: $a^{(0)} = \text{GeLU}(h^{(0)}) \rightarrow (2, 4, 8)$

GPU-1: $a^{(1)} = \text{GeLU}(h^{(1)}) \rightarrow (2, 4, 8)$

Step 4: Row-Parallel W2 (TP Region)

Each GPU computes a partial output:

GPU-0: $y^{(0)} = a^{(0)} \cdot W_2^{(0)} \rightarrow (2, 4, 4) \text{ [partial sum]}$

GPU-1: $y^{(1)} = a^{(1)} \cdot W_2^{(1)} \rightarrow (2, 4, 4) \text{ [partial sum]}$

In vanilla TP, we would ALL-REDUCE: $y = y^{(0)} + y^{(1)} \rightarrow (2, 4, 4)$ on both GPUs

Step 5: REDUCE-SCATTER (TP → SP Transition)

This is where SP differs from vanilla TP! Instead of all-reduce, we reduce-scatter:

REDUCE-SCATTER: sum partials AND scatter along sequence dimension

Let's say the partial outputs are:

GPU-0 partial ($y^{(0)}$):	GPU-1 partial ($y^{(1)}$):
[[0.12, -0.05, 0.08, 0.03],	[[0.04, 0.11, -0.03, 0.07],
[0.07, 0.14, -0.02, 0.09],	[-0.01, 0.06, 0.10, -0.04],
[-0.03, 0.10, 0.05, -0.08],	[0.09, -0.02, 0.04, 0.12],
[0.11, -0.07, 0.13, 0.02]]	[-0.05, 0.08, -0.06, 0.09]]

Step 1: SUM the partials (like reduce phase of all-reduce):

$y_sum = y^{(0)} + y^{(1)}$
= [[0.16, 0.06, 0.05, 0.10], ← token 0
[0.06, 0.20, 0.08, 0.05], ← token 1
[0.06, 0.08, 0.09, 0.04], ← token 2
[0.06, 0.01, 0.07, 0.11]] ← token 3

Step 2: SCATTER along sequence dimension:

GPU-0 receives tokens 0-1: [[0.16, 0.06, 0.05, 0.10],
[0.06, 0.20, 0.08, 0.05]]
Shape: (2, 2, 4) = (B, s/N, h)

GPU-1 receives tokens 2-3: [[0.06, 0.08, 0.09, 0.04],
[0.06, 0.01, 0.07, 0.11]]
Shape: (2, 2, 4)

Communication: same as all-reduce! (reduce + scatter $\approx$ all-reduce internally)

Total reduce-scatter volume: equivalent to all-reduce

Step 6: Dropout (SP Region)

Now dropout operates on the **sharded** sequence — half the memory!

GPU-0: dropout on (2, 2, 4) → tokens 0-1 only

GPU-1: dropout on (2, 2, 4) → tokens 2-3 only

Memory for dropout mask: 16 elements per GPU (was 32 in vanilla TP)

Step 7: Residual Add (SP Region)

The residual connection also operates on sharded tensors:

GPU-0: $x_{out} = x_0 + y_{dropout_0} \rightarrow (2, 2, 4)$

GPU-1: $x_{out} = x_1 + y_{dropout_1} \rightarrow (2, 2, 4)$

This works because:

- x_0 was the SP-region input (tokens 0-1) saved for the residual
- $y_{dropout_0}$ is the SP-region output (tokens 0-1)
- They match!

6. The Embedding Layer with Sequence Parallelism

The embedding layer also benefits from SP. Here's how it's handled:

Vanilla TP (Row-Parallel Embedding):

input_ids: $(B, s) = (2, 4) \rightarrow$ same on all GPUs

embedding lookup: each GPU has full vocab but gets same output

output: $(2, 4, 4) = (B, s, h) \rightarrow$ replicated on all GPUs ✗

TP + SP (Row-Parallel Embedding with Scatter):

input_ids: $(B, s) = (2, 4) \rightarrow$ same on all GPUs

embedding lookup: standard lookup

THEN: reduce-scatter to split sequence!

GPU-0 gets: $(2, 2, 4) \rightarrow$ tokens 0-1

GPU-1 gets: $(2, 2, 4) \rightarrow$ tokens 2-3

Wait, but there's nothing to reduce here (no partial sums from embedding lookup). So we just use **scatter** (or equivalently, each GPU extracts its token slice):

GPU-0: takes $tokens[:, 0:s//N, :] \rightarrow (2, 2, 4)$

GPU-1: takes $tokens[:, s//N:s, :] \rightarrow (2, 2, 4)$

For the output layer (vocab projection), it's column-parallel in vanilla TP. With SP:

Before lm_head:

ALL-GATHER to reconstruct full sequence: $(2, 2, 4) \rightarrow (2, 4, 4)$

lm_head (column-parallel):

GPU-0: $(2, 4, 4) \rightarrow (2, 4, \text{vocab}/N)$

GPU-1: $(2, 4, 4) \rightarrow (2, 4, \text{vocab}/N)$

7. Memory Accounting: Vanilla TP vs TP + SP

Per-GPU Activation Memory for One Transformer Block:

	Vanilla TP	TP + SP	Savings
LayerNorm 1 input	$(B,s,h) = 32$	$(B,s/N,h) = 16$	50%
LayerNorm 1 output	$(B,s,h) = 32$	$(B,s/N,h) = 16$	50%
After all-gather (temp)	—	$(B,s,h) = 32$	(temp)
Q, K, V (in TP region)	$(B,s,h/N) = 16$	$(B,s,h/N) = 16$	same
Attention scores	$(B,H/N,s,s)=16$	$(B,H/N,s,s)=16$	same*
Attention output	$(B,s,h/N) = 16$	$(B,s,h/N) = 16$	same
After W_o (reduce-scatter)	$(B,s,h) = 32$	$(B,s/N,h) = 16$	50%
Residual 1 saved	$(B,s,h) = 32$	$(B,s/N,h) = 16$	50%
LayerNorm 2 input	$(B,s,h) = 32$	$(B,s/N,h) = 16$	50%
LayerNorm 2 output	$(B,s,h) = 32$	$(B,s/N,h) = 16$	50%
After all-gather (temp)	—	$(B,s,h) = 32$	(temp)
FFN intermediate	$(B,s,d_{ff}/N)=64$	$(B,s,d_{ff}/N)=64$	same
After W2 (reduce-scatter)	$(B,s,h) = 32$	$(B,s/N,h) = 16$	50%
Dropout	$(B,s,h) = 32$	$(B,s/N,h) = 16$	50%
Residual 2 saved	$(B,s,h) = 32$	$(B,s/N,h) = 16$	50%
Output	$(B,s,h) = 32$	$(B,s/N,h) = 16$	50%
TOTAL (non-temp)	~400 elements	~272 elements	~32%
* Attention scores: in full SP, these could also be sharded along sequence, but this requires more complex handling of the causal mask.			

For a Real Model (7B, B=8, s=2048, h=4096):

	Vanilla TP (N=8)	TP + SP (N=8)
LayerNorm activations	$B \times s \times h = 64 \text{ MB}$	$B \times (s/N) \times h = 8 \text{ MB}$ ✓
Dropout activations	$B \times s \times h = 64 \text{ MB}$	$B \times (s/N) \times h = 8 \text{ MB}$ ✓

Residual saved	$B \times s \times h = 64 \text{ MB}$	$B \times (s/N) \times h = 8 \text{ MB}$	✓
FFN intermediate	$B \times s \times (d_{\text{ff}}/N) = 88 \text{ MB}$	(same)	
QKV projections	$B \times s \times (h/N) = 8 \text{ MB}$	(same)	
Attention scores	$B \times (H/N) \times s \times s = 128 \text{ MB}$	(same)	
Per-layer total	~416 MB	~248 MB	
Savings:	40%		
32 layers:	~13.3 GB	~7.9 GB	
Savings:	5.4 GB per GPU!		

8. Communication Cost: Identical to Vanilla TP!

This is the beautiful part — SP doesn't add any communication overhead.

Vanilla TP (per transformer block):

After W_o :	ALL-REDUCE	$(B, s, h) = (2, 4, 4) = 32 \text{ elements} \times 2 \text{ bytes} = 64 \text{ B}$
After W_2 :	ALL-REDUCE	$(B, s, h) = (2, 4, 4) = 32 \text{ elements} \times 2 \text{ bytes} = 64 \text{ B}$
Total:	128 bytes per block	

TP + SP (per transformer block):

Before W_q :	ALL-GATHER	$(B, s, h) = 32 \text{ elements} \times 2 = 64 \text{ B}$
After W_o :	REDUCE-SCATTER	$(B, s, h) = 32 \text{ elements} \times 2 = 64 \text{ B}$ ($\approx$ all-reduce)
Before W_1 :	ALL-GATHER	$(B, s, h) = 32 \text{ elements} \times 2 = 64 \text{ B}$
After W_2 :	REDUCE-SCATTER	$(B, s, h) = 32 \text{ elements} \times 2 = 64 \text{ B}$ ($\approx$ all-reduce)
Total:	256 bytes per block ???	

Wait, this looks like $2\times$ the communication! But actually, **all-reduce = reduce-scatter + all-gather** under the hood. So:

Vanilla TP:	All-reduce = reduce-scatter + all-gather (fused into one operation)
TP + SP:	reduce-scatter (explicit) + all-gather (explicit)
Same operations, just separated in time to enable SP region in between!	

The communication volume is **exactly the same**. We're just "unrolling" the all-reduce into its constituent parts and inserting computation (LayerNorm, Dropout) between them.

9. Implementation: The Key Code Changes

Vanilla TP (Row-Parallel Linear):

```
python

class RowParallelLinear(nn.Module):
    def forward(self, x):
        out = F.linear(x, self.weight, self.bias)
        dist.all_reduce(out) # Sum partials across GPUs
        return out
```

TP + SP (Row-Parallel Linear with Reduce-Scatter):

```
python

class RowParallelLinearSP(nn.Module):
    def forward(self, x):
        out = F.linear(x, self.weight, self.bias)
        # Replace all-reduce with reduce-scatter
        out = reduce_scatter(out, dim=1) # Scatter along sequence dimension
        return out # Shape: (B, s/N, h) instead of (B, s, h)
```

TP + SP (Column-Parallel Linear with All-Gather):

```
python

class ColumnParallelLinearSP(nn.Module):
    def forward(self, x):
        # x is in SP region: (B, s/N, h)
        x = all_gather(x, dim=1) # Reconstruct full sequence: (B, s, h)
        out = F.linear(x, self.weight, self.bias)
        return out # Shape: (B, s, h/N) — now in TP region
```

Transformer Block with SP:

```
python
```

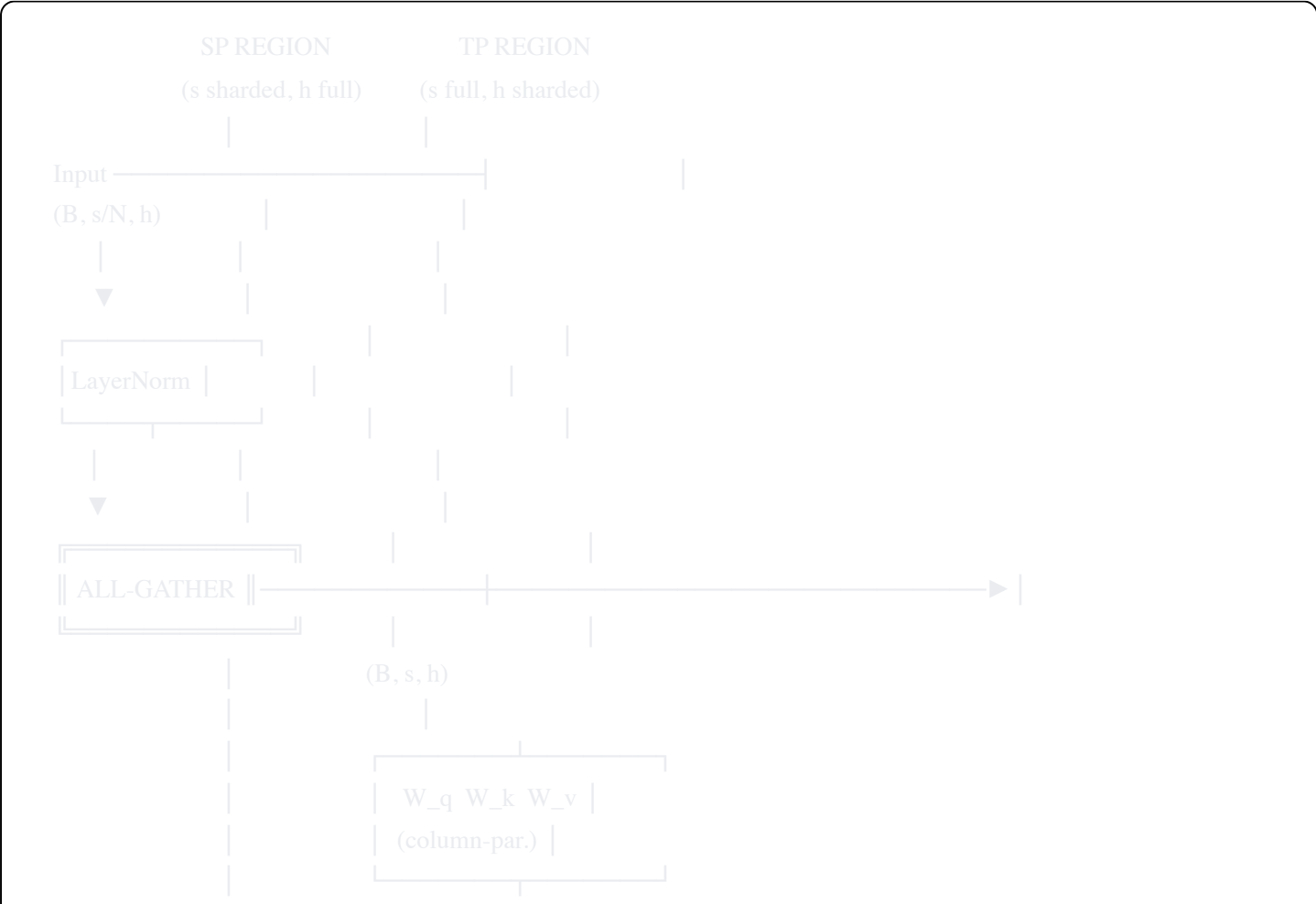
```
class TransformerBlockSP(nn.Module):
    def forward(self, x):
        # x enters in SP region: (B, s/N, h)
        residual = x

        x = self.ln1(x)          # (B, s/N, h) — SP region
        x = all_gather(x, dim=1)  # (B, s, h) — transition to TP
        x = self.attention(x)     # (B, s, h) — TP region (includes reduce-scatter at end)
                                # → (B, s/N, h) — back to SP
        x = x + residual          # (B, s/N, h) — SP region

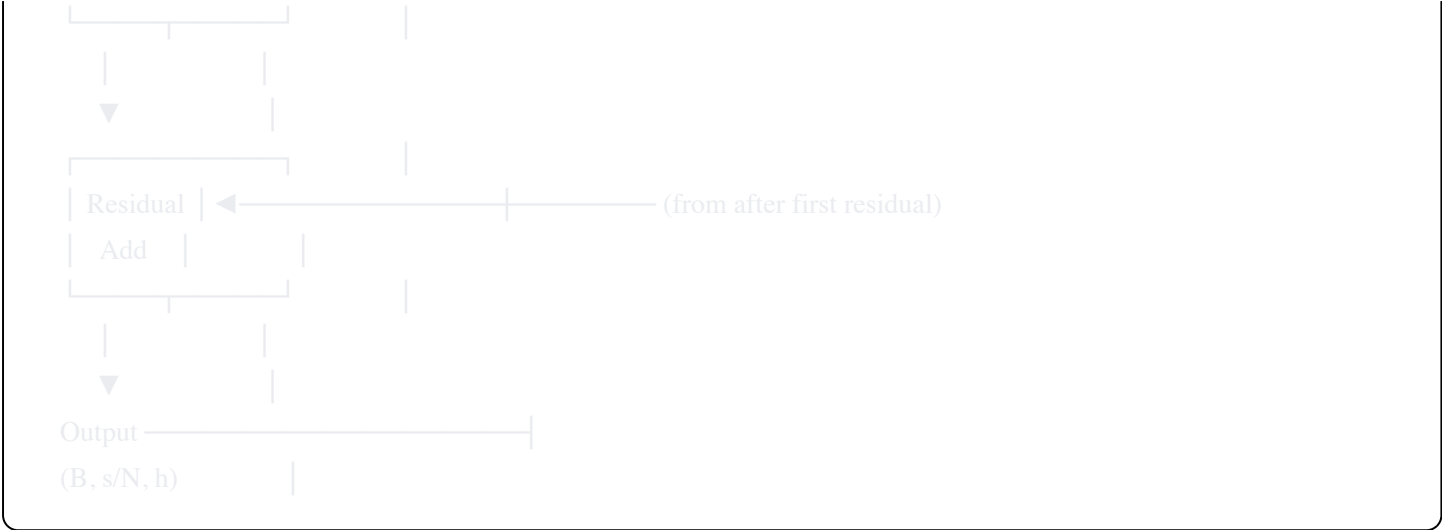
        residual = x
        x = self.ln2(x)          # (B, s/N, h) — SP region
        x = all_gather(x, dim=1)  # (B, s, h) — transition to TP
        x = self.ffn(x)           # (B, s, h) — TP region (includes reduce-scatter at end)
                                # → (B, s/N, h) — back to SP
        x = self.dropout(x)      # (B, s/N, h) — SP region
        x = x + residual          # (B, s/N, h) — SP region

    return x # (B, s/N, h) — stays in SP region for next block
```

10. Visual Summary: Data Flow Through the Block







11. Summary Table: Activation Shapes by Region

ACTIVATION SHAPES IN TP + SP			
REGION	OPERATION	SHAPE	PARALLELISM
SP Region	Input to block	(B, s/N, h)	sequence split
	LayerNorm	(B, s/N, h)	sequence split
	Residual tensors	(B, s/N, h)	sequence split
	Dropout	(B, s/N, h)	sequence split
SP → TP	ALL-GATHER	(B, s/N, h)→(B, s, h)	
Transition	Reconstructs full sequence for matmul		
TP Region	After col-par (QKV)	(B, s, h/N)	hidden split
	Attention scores	(B, H/N, s, s)	heads split
	After row-par (W_o)	(B, s, h) partial	needs reduction
	FFN intermediate	(B, s, d_ff/N)	d_ff split
TP → SP	REDUCE-SCATTER	(B, s, h)→(B, s/N, h)	
Transition	Sums partials AND scatters to SP region		
MEMORY SAVINGS (vs vanilla TP):			
• LayerNorm activations: 1/N of vanilla TP			
• Dropout activations: 1/N of vanilla TP			

• Residual stored tensors:	1/N of vanilla TP
• TP region activations:	same as vanilla TP
COMMUNICATION OVERHEAD:	
• Same as vanilla TP (all-reduce = reduce-scatter + all-gather)	

12. When to Use Sequence Parallelism

ALWAYS use SP when using TP.
There is essentially NO downside:
✓ Same communication volume as vanilla TP
✓ 30-50% reduction in activation memory
✓ Enables larger batch sizes or longer sequences
✓ Implemented in all major frameworks (Megatron-LM, DeepSpeed, etc.)
The only "cost" is implementation complexity, which is already handled by the frameworks you're using.
SP is standard practice for any serious large-scale training.

The Full Picture: Combining All Parallelism Strategies

Modern Large-Scale Training (e.g., LLaMA-70B):	
Tensor Parallelism (TP=8)	Within each node (NVLink)
+ Sequence Parallelism	Always paired with TP
Pipeline Parallelism (PP=4)	Across nodes in a pipeline
Data Parallelism (DP=16)	Across pipeline replicas
+ ZeRO Stage 1	Partition optimizer states
Total: 8 × 4 × 16 = 512 GPUs	

Memory per GPU:

- Parameters: $70\text{B} / 8 / 4 \approx 2.2\text{B params} = 4.4\text{ GB (BF16)}$
- Activations: reduced by TP+SP ($\sim 2\text{ GB}$ per microbatch)
- Optimizer: $70\text{B} \times 12 / 8 / 4 / 16 \approx 1.3\text{ GB}$
- Total: $\sim 8\text{-}10\text{ GB}$, fits comfortably on 80GB A100/H100

